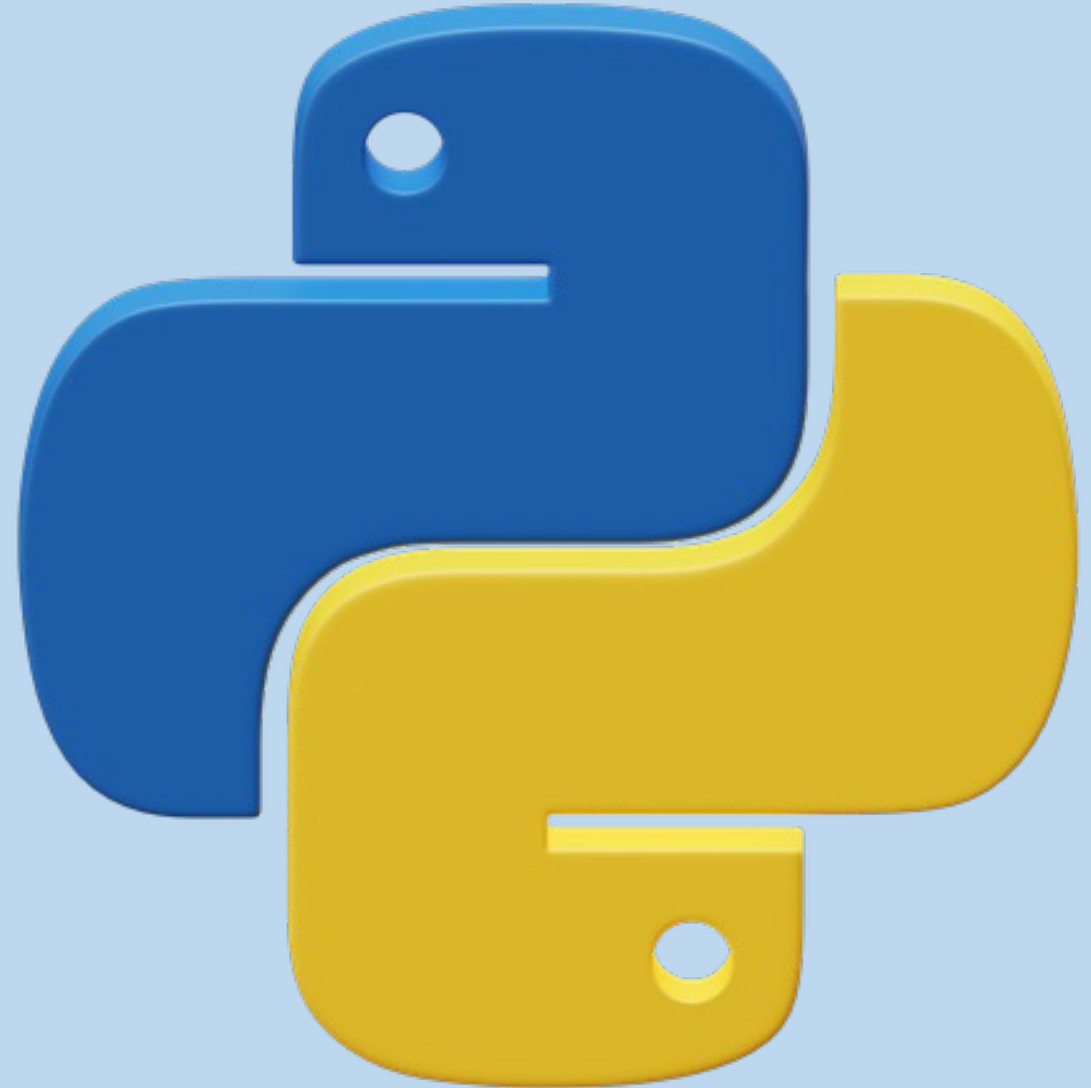


Advanced Programming

Reference Project



Today's Topics

Project Structure

- Why

Pizzaria app structure

- Packages
- Files

N-Tier Architecture

- Presentation Layer
- Logic Layer
- Data Layer

Design Patterns in Pizzaria app

- Some examples

Next steps

- Recommendations for your next steps

Project Structure

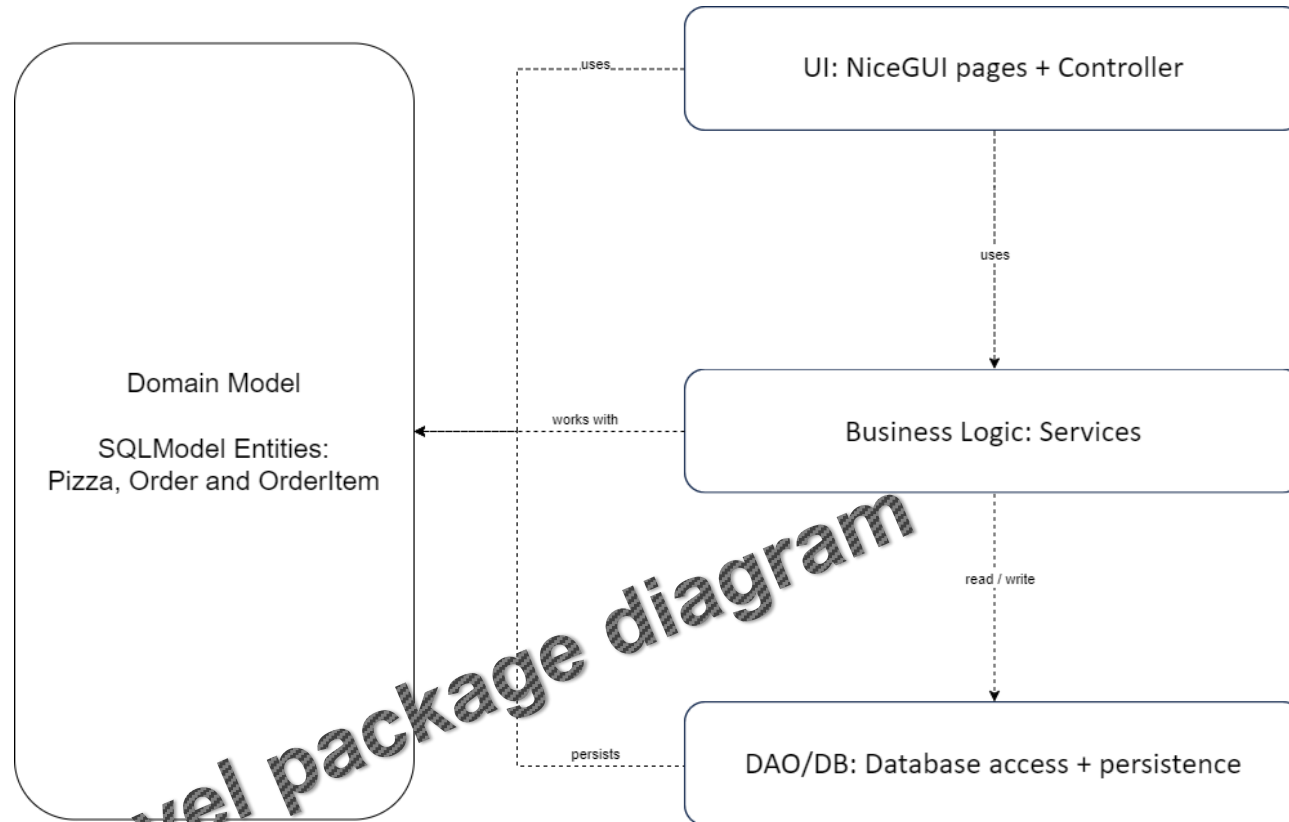
A project structure divides a program into **logical subsystems**.

A well-designed project structure **increases cohesion** and **reduces coupling** between parts of an application. This means:

- each subsystem focuses on its **specific responsibility**
- dependencies between subsystems are kept **as low as possible**

DRY - KISS

Pizzeria app structure



```

pizza_app/
├── __init__.py
├── __main__.py
├── application.py
├── data_access/
│   ├── __init__.py
│   ├── dao.py
│   ├── db.py
│   └── seed.py
├── domain/
│   ├── __init__.py
│   └── models.py
├── services/
│   ├── __init__.py
│   ├── invoice_service.py
│   ├── order_service.py
│   ├── pizza_service.py
│   └── pricing_service.py
└── ui/
    ├── __init__.py
    ├── controllers.py
    └── pages.py
  
```

N-Tier Architecture

Presentation Layer

The top layer is the interface (GUI / CLI / API)

The main responsibility of this layer is:

- translating user input into actions
- displaying results to the user

```
2 Margherita
Sub Total: 29.00
Discount: 0.00
Total: 29.00
```

Confirmation

Shopping
Controller

Logic Layer (Business / Services)

This layer implements the **business logic**.

The main responsibilities are:

- implementing use cases
- performing calculations
- coordinating the application flow

OrderService

InvoiceService

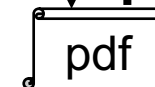
Data Layer (data_access)

This layer handles **data access**.

The main responsibilities are:

- storing and retrieving data

OrderDAO



Model (Domain / Data)

The business domain is typically implemented in a separate layer, because the same entities are used across all layers.

These objects:

- represent real-world concepts
- are used in UI, logic, and data layers

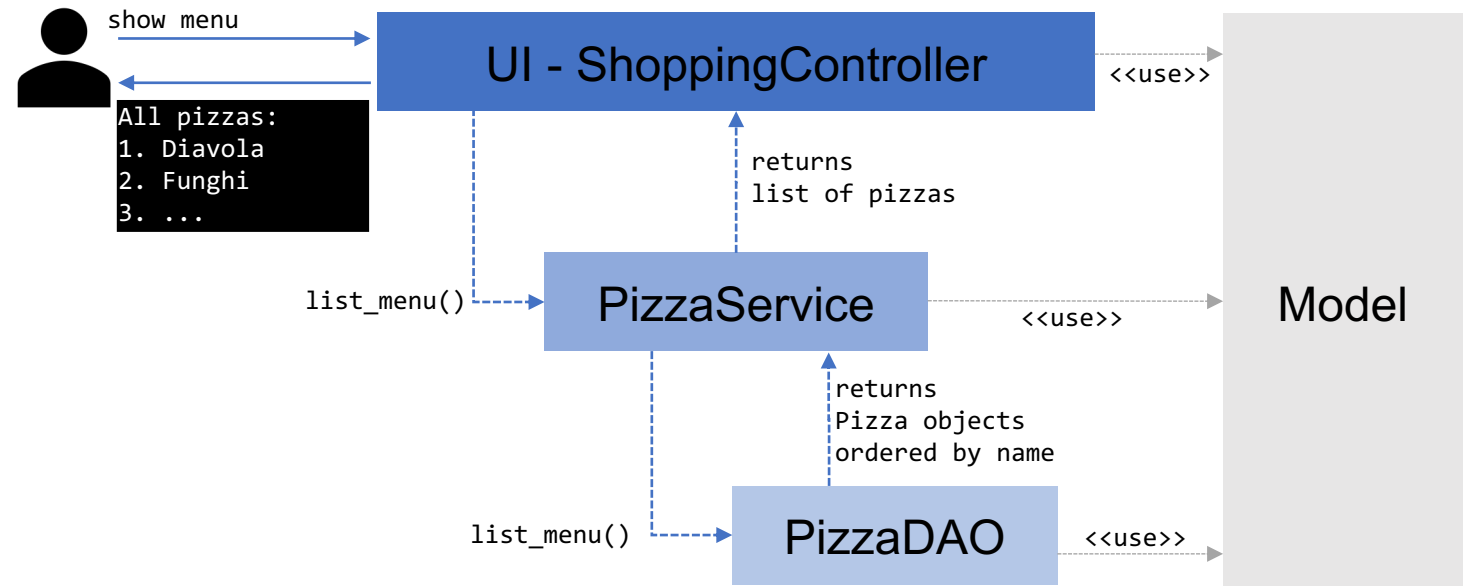


Calls go down (methods),
data comes back up (returns)

N-Tier Architecture

```

pizza_app/
├── __init__.py
├── __main__.py
├── application.py
├── data_access/
│   ├── __init__.py
│   ├── dao.py
│   ├── db.py
│   └── seed.py
├── domain/
│   ├── __init__.py
│   └── models.py
├── services/
│   ├── __init__.py
│   ├── invoice_service.py
│   ├── order_service.py
│   ├── pizza_service.py
│   └── pricing_service.py
└── ui/
    ├── __init__.py
    ├── controllers.py
    └── pages.py
    
```



Design Patterns in the Pizzeria app

- Data Access Object (DAO)
 - DAO layer sits between: business logic and database / ORM
 - It translates “what the app wants” into “how to query/store it”
 - All database details are hidden inside DAOs
- Facade (see db.py)
 - Database class becomes a simplified single place that controls the DB setup, i.e. engine creation, schema setup, seeding, session lifecycle
- Model – View – Controller (ui/services/data_access)
 - Here implemented as View → Controller → Service → DAO → DB/Models

The next steps ...

- Fix the user stories in the team and with the coach (Defining the scope)
- Create a high-level package diagram (Defining the main building blocks)
- Define the services (from the user stories)
- Create an ERD (and a data model, see lecture about ORM)
- Decide about Design Patterns
(find out what brings an advantage, see lecture about Design Patterns)
- Decide who makes what and clarify the collaboration model